

Hoare Logic I

Triples and rules

Cristina David

Lecture 8

COMSM0067 — Advanced Topics in Programming Languages

University of Bristol

Where we have been

- **Statics.** Typing judgements $\vdash e : \sigma$.
- **Dynamics.** Transitions $e \mapsto e'$ and $e \mapsto^* e'$.
- **Type safety.** Progress and preservation.

Where we are going

Type safety says: **well-typed programs do not get stuck.**

That is a real theorem, and your first **verification** result. But what does it *not* say?

Where we are going

Type safety says: **well-typed programs do not get stuck.**

That is a real theorem, and your first **verification** result. But what does it *not* say?

$\text{Int} \rightarrow \text{Int}$ is inhabited by:

- the doubling function
- the constant 0
- the function computing the n th prime

Type safety cannot tell them apart.

Where we are going

Type safety says: **well-typed programs do not get stuck.**

That is a real theorem, and your first **verification** result. But what does it *not* say?

$\text{Int} \rightarrow \text{Int}$ is inhabited by:

- the doubling function
- the constant 0
- the function computing the n th prime

Type safety cannot tell them apart.

Today: a **third semantics** that can.

The gap

Statics: a *coarse* specification

It says a program will not go wrong. Never **which** function it computes.

The gap

Statics: a *coarse* specification

It says a program will not go wrong. Never **which** function it computes.

Dynamics: not a specification at all

The *ground truth* of execution. It says what *happens*, not what **should** happen.

The gap

Statics: a *coarse* specification

It says a program will not go wrong. Never **which** function it computes.

Dynamics: not a specification at all

The *ground truth* of execution. It says what *happens*, not what **should** happen.

So

We cannot *state* what a program is supposed to compute, and therefore cannot *prove* that it does.

Axiomatic semantics is where “verification” stops meaning “it won’t crash” and starts meaning “it computes the right answer”.

The language: Imp

We change vehicles

The λ -calculus hid the thing we now care about: **mutable state**.

Hoare logic reasons about **how state changes**. So it needs a language with some **state-changing, effectful construct** for the reasoning to bite on.

We change vehicles

The λ -calculus hid the thing we now care about: **mutable state**.

Hoare logic reasons about **how state changes**. So it needs a language with some **state-changing, effectful construct** for the reasoning to bite on.

We pick the simplest one: **assignment**, which is the state-changer in Imp.

Imp — the minimal imperative language.

Everything to the end of the unit lives in Imp (plus, later, a heap).

Imp: syntax

$a ::= n \mid x \mid a + a \mid a - a \mid a * a$ arithmetic expressions

$b ::= \text{true} \mid \text{false} \mid a = a \mid a \leq a \mid \neg b \mid b \wedge b$ boolean expressions

$c ::= \text{skip}$ commands

| $x := a$

| $c ; c$

| if b then c else c

| while b do c

Two sorts, **int** and **bool**, kept apart by the grammar, so no separate typing judgement,

and nothing gets stuck. No functions, no heap. **Assignment is the point.**

Imp: states

A **state** is a total map from variables to integers:

$$\sigma : \text{Var} \rightarrow \mathbb{Z}$$

Expressions denote functions of the state:

$$\llbracket a \rrbracket \sigma \in \mathbb{Z} \qquad \llbracket b \rrbracket \sigma \in \{tt, ff\}$$

Update notation: $\sigma[x \mapsto v]$ is σ with x remapped to v .

Variables hold integers only — a **boolean cannot live in the state**. Assignment is $x := a$, and booleans appear only as guards, evaluated on demand and never stored.

A notation warning

σ changes meaning today

Previously, σ ranged over **types**: $\vdash e : \sigma$.

From now on, σ ranges over **states**.

This is the standard convention in the verification literature.

Small-step, or big-step?

Previously, the dynamics was **small-step**:

$$e \mapsto e' \qquad e \mapsto^* e'$$

Today it will be **big-step**:

$$\langle c, \sigma \rangle \Downarrow \sigma'$$

straight from a command to its *final state*, saying nothing about the way there.

Small-step, or big-step?

Previously, the dynamics was **small-step**:

$$e \mapsto e' \qquad e \mapsto^* e'$$

Today it will be **big-step**:

$$\langle c, \sigma \rangle \Downarrow \sigma'$$

straight from a command to its *final state*, saying nothing about the way there.

They agree: $\langle c, \sigma \rangle \Downarrow \sigma' \quad \text{iff} \quad \langle c, \sigma \rangle \mapsto^* \langle \text{skip}, \sigma' \rangle$

Small-step, or big-step?

Previously, the dynamics was **small-step**:

$$e \mapsto e' \qquad e \mapsto^* e'$$

Today it will be **big-step**:

$$\langle c, \sigma \rangle \Downarrow \sigma'$$

straight from a command to its *final state*, saying nothing about the way there.

$$\text{They agree: } \langle c, \sigma \rangle \Downarrow \sigma' \quad \text{iff} \quad \langle c, \sigma \rangle \mapsto^* \langle \text{skip}, \sigma' \rangle$$

Why big-step? Because a specification constrains only the **initial** and **final** states, and that is exactly the relation $\Downarrow$ gives us.

Imp: dynamics (big-step)

$$\begin{array}{c} \frac{}{\langle \text{skip}, \sigma \rangle \Downarrow \sigma} \quad \frac{}{\langle x := a, \sigma \rangle \Downarrow \sigma[x \mapsto \llbracket a \rrbracket \sigma]} \quad \frac{\langle c_1, \sigma \rangle \Downarrow \sigma'' \quad \langle c_2, \sigma'' \rangle \Downarrow \sigma'}{\langle c_1; c_2, \sigma \rangle \Downarrow \sigma'} \\[10pt] \frac{\llbracket b \rrbracket \sigma = tt \quad \langle c_1, \sigma \rangle \Downarrow \sigma'}{\langle \text{if } b \text{ then } c_1 \text{ else } c_2, \sigma \rangle \Downarrow \sigma'} \quad \frac{\llbracket b \rrbracket \sigma = ff \quad \langle c_2, \sigma \rangle \Downarrow \sigma'}{\langle \text{if } b \text{ then } c_1 \text{ else } c_2, \sigma \rangle \Downarrow \sigma'} \\[10pt] \frac{\llbracket b \rrbracket \sigma = ff}{\langle \text{while } b \text{ do } c, \sigma \rangle \Downarrow \sigma} \quad \frac{\llbracket b \rrbracket \sigma = tt \quad \langle c, \sigma \rangle \Downarrow \sigma'' \quad \langle \text{while } b \text{ do } c, \sigma'' \rangle \Downarrow \sigma'}{\langle \text{while } b \text{ do } c, \sigma \rangle \Downarrow \sigma'} \end{array}$$

Triples

Assertions

An **assertion** P is a predicate on states — a first-order formula over the program variables.

Examples:

$x > 0$ $x = y + 1$ $\exists k. x = 2k$ True False

Write $\sigma \models P$ for “ P holds in state σ ”.

Assertions

An **assertion** P is a predicate on states — a first-order formula over the program variables.

Examples:

$x > 0$ $x = y + 1$ $\exists k. x = 2k$ True False

Write $\sigma \models P$ for “ P holds in state σ ”.

Note

Assertions are *not* boolean expressions of Imp. They may use quantifiers, they are the language of *mathematics*, not of the program.

$$\{P\} \ c \ \{Q\}$$

$$\{P\} \ c \ \{Q\}$$

Reading

If P holds before executing c ,
and c terminates,
then Q holds afterwards.

$$\{P\} \ c \ \{Q\}$$

Reading

If P holds before executing c ,
and c terminates,
then Q holds afterwards.

P is the **precondition**. Q is the **postcondition**.

This is called **partial correctness**. It says **nothing** about whether c terminates.

Non-termination example

$\{P\}$ while true do skip $\{Q\}$

Non-termination example

$\{P\}$ while true do skip $\{Q\}$

This is valid for every P and every Q .

The loop never terminates. So “if it terminates then Q ” is vacuously true.

Non-termination example

$\{P\}$ while true do skip $\{Q\}$

This is valid for every P and every Q .

The loop never terminates. So “if it terminates then Q ” is vacuously true.

A program that does nothing, forever, “satisfies” every specification you can write.

Total correctness (which we don’t cover) exists to close exactly this hole.

Which of these are valid?

1. $\{\text{True}\} \ x := 5 \ \{x = 5\}$
2. $\{\text{True}\} \ x := 5 \ \{x = 6\}$
3. $\{x = 6\} \ x := 5 \ \{x = 5\}$

Quick check

Which of these are valid?

1. $\{\text{True}\} \ x := 5 \ \{x = 5\}$
2. $\{\text{True}\} \ x := 5 \ \{x = 6\}$
3. $\{x = 6\} \ x := 5 \ \{x = 5\}$

Yes / No / Yes.

Making “valid” precise

We have been saying “valid” informally. The definition is just the triple’s reading, in terms of the dynamics.

Validity

$\models \{P\} c \{Q\}$ iff for all σ, σ' :

$\sigma \models P$ and $\langle c, \sigma \rangle \Downarrow \sigma'$ implies $\sigma' \models Q$.

Making “valid” precise

We have been saying “valid” informally. The definition is just the triple’s reading, in terms of the dynamics.

Validity

$\models \{P\} c \{Q\}$ iff for all σ, σ' :

$\sigma \models P$ and $\langle c, \sigma \rangle \Downarrow \sigma'$ implies $\sigma' \models Q$.

The rules

The proof system

We now define a judgement $\vdash \{P\} c \{Q\}$, read “**derivable**”: the triple can be derived using the rules below.

How this relates to **validity** is the last question of this section.

Six rules. One per syntactic form, plus one more.

The proof system

We now define a judgement $\vdash \{P\} c \{Q\}$, read “**derivable**”: the triple can be derived using the rules below.

How this relates to **validity** is the last question of this section.

Six rules. One per syntactic form, plus one more.

The last one is the interesting one.

$$\frac{}{\vdash \{P\} \text{ skip } \{P\}} \text{ Skip}$$

No premises, so this is an **axiom**.

$$\frac{}{\vdash \{P\} \text{ skip } \{P\}} \text{ Skip}$$

No premises, so this is an **axiom**.

$$\frac{\vdash \{P\} c_1 \{R\} \quad \vdash \{R\} c_2 \{Q\}}{\vdash \{P\} c_1; c_2 \{Q\}} \text{ Seq}$$

The midpoint assertion R is the **glue**. Finding it is the whole work of a sequential proof

— and, as we will see, the assignment axiom will *compute* it for us.

$$\frac{\vdash \{P \wedge b\} c_1 \{Q\} \quad \vdash \{P \wedge \neg b\} c_2 \{Q\}}{\vdash \{P\} \text{ if } b \text{ then } c_1 \text{ else } c_2 \{Q\}} \text{ If}$$

Each branch may **assume the guard's truth value**.

Both branches must establish the **same** Q .

The rule **merges** the two branches back together.

Assignment — the surprising one

$$\frac{}{\vdash \{Q[a/x]\} x := a \{Q\}} \text{Assign}$$

Assignment — the surprising one

$$\frac{}{\vdash \{Q[a/x]\} x := a \{Q\}} \text{Assign}$$

Read it right to left.

To guarantee Q *after* $x := a$,
demand that Q -with- a -in-place-of- x holds *before*.

Assignment: micro-examples

Want	Command	Axiom gives
$x > 0$	$x := x + 1$	$x + 1 > 0$, i.e. $x > -1$
$x = 5$	$x := 5$	$5 = 5$, i.e. True
$x = y$	$x := y$	$y = y$, i.e. True
$x = 6$	$x := 5$	$5 = 6$, i.e. False

Assignment: micro-examples

Want	Command	Axiom gives
$x > 0$	$x := x + 1$	$x + 1 > 0$, i.e. $x > -1$
$x = 5$	$x := 5$	$5 = 5$, i.e. True
$x = y$	$x := y$	$y = y$, i.e. True
$x = 6$	$x := 5$	$5 = 6$, i.e. False

The last one is the **sanity check**.

You cannot land in $x = 6$ by assigning 5. The axiom gives you False, the only precondition from which the triple holds, and one you can never satisfy.

Loops — the invariant rule

$$\frac{\vdash \{I \wedge b\} c \{I\}}{\vdash \{I\} \text{ while } b \text{ do } c \{I \wedge \neg b\}} \text{ While}$$

I is the **loop invariant**:

- true on entry,
- **preserved** by the body (that is the premise),
- hence true on exit — and on exit, the guard has **failed**.

Loops — the invariant rule

$$\frac{\vdash \{I \wedge b\} c \{I\}}{\vdash \{I\} \text{ while } b \text{ do } c \{I \wedge \neg b\}} \text{ While}$$

I is the **loop invariant**:

- true on entry,
- **preserved** by the body (that is the premise),
- hence true on exit — and on exit, the guard has **failed**.

Where is I from?

The rule does not tell you. **Finding I is the creative core of verification**, and it is the whole of next lecture.

Consequence — where logic enters

$$\frac{P \Rightarrow P' \quad \vdash \{P'\} c \{Q'\} \quad Q' \Rightarrow Q}{\vdash \{P\} c \{Q\}} \text{Conseq}$$

The other five rules are **syntax-directed**: look at c , apply the rule.

This one is **not**. It can be applied anywhere, any time.

Consequence — where logic enters

$$\frac{P \Rightarrow P' \quad \vdash \{P'\} c \{Q'\} \quad Q' \Rightarrow Q}{\vdash \{P\} c \{Q\}} \text{Conseq}$$

The other five rules are **syntax-directed**: look at c , apply the rule.

This one is **not**. It can be applied anywhere, any time. Its premises $P \Rightarrow P'$ and $Q' \Rightarrow Q$ are entailments in the **assertion logic**. They are discharged by *ordinary mathematics* or e.g. an **SMT solver**.

This is the seam between program reasoning and logical reasoning.

Consequence: mind the direction

$$P \Rightarrow P'$$

$$Q' \Rightarrow Q$$

Strengthen the precondition.

Weaken the postcondition.

Consequence: mind the direction

$$P \Rightarrow P'$$

$$Q' \Rightarrow Q$$

Strengthen the precondition.

Weaken the postcondition.

You may always **assume more** and **promise less**.

You may **never** assume less and promise more.

The six rules, together

$$\frac{}{\{P\} \text{ skip } \{P\}} \text{ Skip}$$

$$\frac{}{\{Q[a/x]\} x := a \{Q\}} \text{ Assign}$$

$$\frac{\{P\} c_1 \{R\} \quad \{R\} c_2 \{Q\}}{\{P\} c_1; c_2 \{Q\}} \text{ Seq}$$

$$\frac{\{P \wedge b\} c_1 \{Q\} \quad \{P \wedge \neg b\} c_2 \{Q\}}{\{P\} \text{ if } b \text{ then } c_1 \text{ else } c_2 \{Q\}} \text{ If}$$

$$\frac{\{I \wedge b\} c \{I\}}{\{I\} \text{ while } b \text{ do } c \{I \wedge \neg b\}} \text{ While}$$

$$\frac{P \Rightarrow P' \quad \{P'\} c \{Q'\} \quad Q' \Rightarrow Q}{\{P\} c \{Q\}} \text{ Conseq}$$

That is the whole logic.

Do the rules tell the truth?

We now have two notions: **valid** ($\models$, about execution) and **derivable** ($\vdash$, about proofs).
How do they relate?

Do the rules tell the truth?

We now have two notions: **valid** ($\models$, about execution) and **derivable** ($\vdash$, about proofs).
How do they relate?

Soundness: if $\vdash \{P\} c \{Q\}$ then $\models \{P\} c \{Q\}$

Everything we can derive is true of execution. Check each rule against the big-step semantics. **The rules never lie.**

Do the rules tell the truth?

We now have two notions: **valid** ($\models$, about execution) and **derivable** ($\vdash$, about proofs).
How do they relate?

Soundness: if $\vdash \{P\} c \{Q\}$ then $\models \{P\} c \{Q\}$

Everything we can derive is true of execution. Check each rule against the big-step semantics. **The rules never lie.**

Completeness: if $\models \{P\} c \{Q\}$ then $\vdash \{P\} c \{Q\}$?

Not quite. Conseq asks us to prove facts about integers, and by Gödel some true facts of arithmetic have no proof.

Do the rules tell the truth?

We now have two notions: **valid** ($\models$, about execution) and **derivable** ($\vdash$, about proofs). How do they relate?

Soundness: if $\vdash \{P\} c \{Q\}$ then $\models \{P\} c \{Q\}$

Everything we can derive is true of execution. Check each rule against the big-step semantics. **The rules never lie.**

Completeness: if $\models \{P\} c \{Q\}$ then $\vdash \{P\} c \{Q\}$?

Not quite. Conseq asks us to prove facts about integers, and by Gödel some true facts of arithmetic have no proof.

But that is the **only** gap: given every true implication, every valid triple is derivable (**relative completeness**, Cook 1978).

Incompleteness lives in the assertion logic, never in the six rules.

A worked example

$$\{x = A \wedge y = B\} \quad t := x; \quad x := y; \quad y := t \quad \{x = B \wedge y = A\}$$

A and B are **logical variables** (auxiliary variables).

- They do **not** appear in the program.
- They are universally quantified constants that **name the initial values**.
- Without them we could not *state* what swapping means.

The swap: work backwards

Start from the postcondition $Q \equiv (x = B \wedge y = A)$ and push it up through each assignment.

The swap: work backwards

Start from the postcondition $Q \equiv (x = B \wedge y = A)$ and push it up through each assignment.

$$\text{3rd: } y := t \quad \text{pre} = Q[t/y] \quad = (x = B \wedge t = A)$$

The swap: work backwards

Start from the postcondition $Q \equiv (x = B \wedge y = A)$ and push it up through each assignment.

$$\text{3rd: } y := t \quad \text{pre} = Q[t/y] \quad = (x = B \wedge t = A)$$

$$\text{2nd: } x := y \quad \text{pre} = (\dots)[y/x] \quad = (y = B \wedge t = A)$$

The swap: work backwards

Start from the postcondition $Q \equiv (x = B \wedge y = A)$ and push it up through each assignment.

$$\text{3rd: } y := t \quad \text{pre} = Q[t/y] \quad = (x = B \wedge t = A)$$

$$\text{2nd: } x := y \quad \text{pre} = (\dots)[y/x] \quad = (y = B \wedge t = A)$$

$$\text{1st: } t := x \quad \text{pre} = (\dots)[x/t] \quad = (y = B \wedge x = A)$$

The swap: work backwards

Start from the postcondition $Q \equiv (x = B \wedge y = A)$ and push it up through each assignment.

$$\text{3rd: } y := t \quad \text{pre} = Q[t/y] \quad = (x = B \wedge t = A)$$

$$\text{2nd: } x := y \quad \text{pre} = (\dots)[y/x] \quad = (y = B \wedge t = A)$$

$$\text{1st: } t := x \quad \text{pre} = (\dots)[x/t] \quad = (y = B \wedge x = A)$$

By three Assign axioms and two Seq steps:

$$\vdash \{y = B \wedge x = A\} \quad t := x; \quad x := y; \quad y := t \quad \{x = B \wedge y = A\}$$

The swap: work backwards

Start from the postcondition $Q \equiv (x = B \wedge y = A)$ and push it up through each assignment.

$$\text{3rd: } y := t \quad \text{pre} = Q[t/y] \quad = (x = B \wedge t = A)$$

$$\text{2nd: } x := y \quad \text{pre} = (\dots)[y/x] \quad = (y = B \wedge t = A)$$

$$\text{1st: } t := x \quad \text{pre} = (\dots)[x/t] \quad = (y = B \wedge x = A)$$

By three Assign axioms and two Seq steps:

$$\vdash \{y = B \wedge x = A\} \quad t := x; \quad x := y; \quad y := t \quad \{x = B \wedge y = A\}$$

And Conseq closes it: $(x = A \wedge y = B) \Rightarrow (y = B \wedge x = A)$. ■

What just happened

We never guessed anything.

Assign **computed** each midpoint assertion.

Seq **threaded** them together.

Conseq only **tidied** the entry condition.

The proof writes itself, right to left.

What just happened

We never guessed anything.

Assign **computed** each midpoint assertion.

Seq **threaded** them together.

Conseq only **tidied** the entry condition.

The proof writes itself, right to left.

No accident: the assignment axiom runs backwards, so **verification is a backward calculation**. Next lecture we name it: the **weakest precondition**.

What just happened

We never guessed anything.

Assign **computed** each midpoint assertion.

Seq **threaded** them together.

Conseq only **tidied** the entry condition.

The proof writes itself, right to left.

No accident: the assignment axiom runs backwards, so **verification is a backward calculation**. Next lecture we name it: the **weakest precondition**. And then we meet While, which *breaks* the mechanical flow: no rule computes an invariant for you.

Wrap

Today

- A **third semantics**: not what a program does, but what it *should* do.
- The **Hoare triple** $\{P\} c \{Q\}$, which is **partial** correctness, so a diverging program satisfies everything.
- **Six rules**. Five syntax-directed; Conseq is the seam where logic enters.
- The **assignment axiom runs backwards** and so, therefore, does verification. The swap proof **wrote itself**.

Today

- A **third semantics**: not what a program does, but what it *should* do.
- The **Hoare triple** $\{P\} c \{Q\}$, which is **partial** correctness, so a diverging program satisfies everything.
- **Six rules**. Five syntax-directed; Conseq is the seam where logic enters.
- The **assignment axiom runs backwards** and so, therefore, does verification. The swap proof **wrote itself**.

Six rules is the *whole* logic. Everything from here is about **using** them.